

A vertical decorative strip on the left side of the slide, featuring a close-up of autumn leaves in shades of red, orange, and yellow.

Polymorphism



Polymorphism

- **Polymorphism is an object-oriented concept that allows us to create versatile software designs**

Outline



Polymorphic References

Polymorphism via Inheritance

Polymorphism via Interfaces

Binding

- Consider the following method invocation:

`obj.doIt();`

- At some point, this invocation is *bound* to the definition of the method that it invokes
- If this binding occurred at compile time, then that line of code would call the same method every time
- However, Java defers method binding until run time -- this is called *dynamic binding* or *late binding*
- Late binding provides flexibility in program design



Polymorphism

- The term *polymorphism* literally means "having many forms"
- A *polymorphic reference* is a variable that can refer to different types of objects at different points in time
- The method invoked through a polymorphic reference can change from one invocation to the next
- All object references in Java are potentially polymorphic

Polymorphism

- Suppose we create the following reference variable:

`Occupation job;`

- Java allows this reference to point to an `Occupation` object, or to any object of any compatible type
- This compatibility can be established using inheritance or using interfaces
- Careful use of polymorphic references can lead to elegant, robust software designs

Outline

Polymorphic References

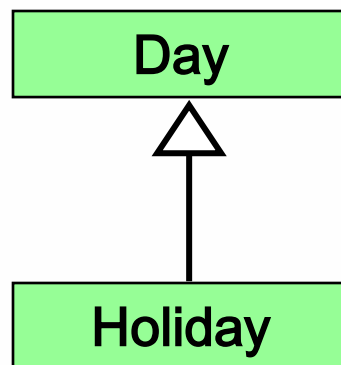


Polymorphism via Inheritance

Polymorphism via Interfaces

References and Inheritance

- An object reference can refer to an object of its class, or to an object of any class related to it by inheritance
- For example, if the `Holiday` class is derived from a class called `Day`, then a `Day` reference could be used to point to a `Holiday` object



```
Day day;  
day = new Holiday();
```




References and Inheritance

- **Assigning a child object to a parent reference is considered to be a widening conversion, and can be performed by simple assignment**
- **Assigning an parent object to a child reference can be done also, but it is considered a narrowing conversion and must be done with a cast**
- **The widening conversion is the most useful**

Polymorphism via Inheritance

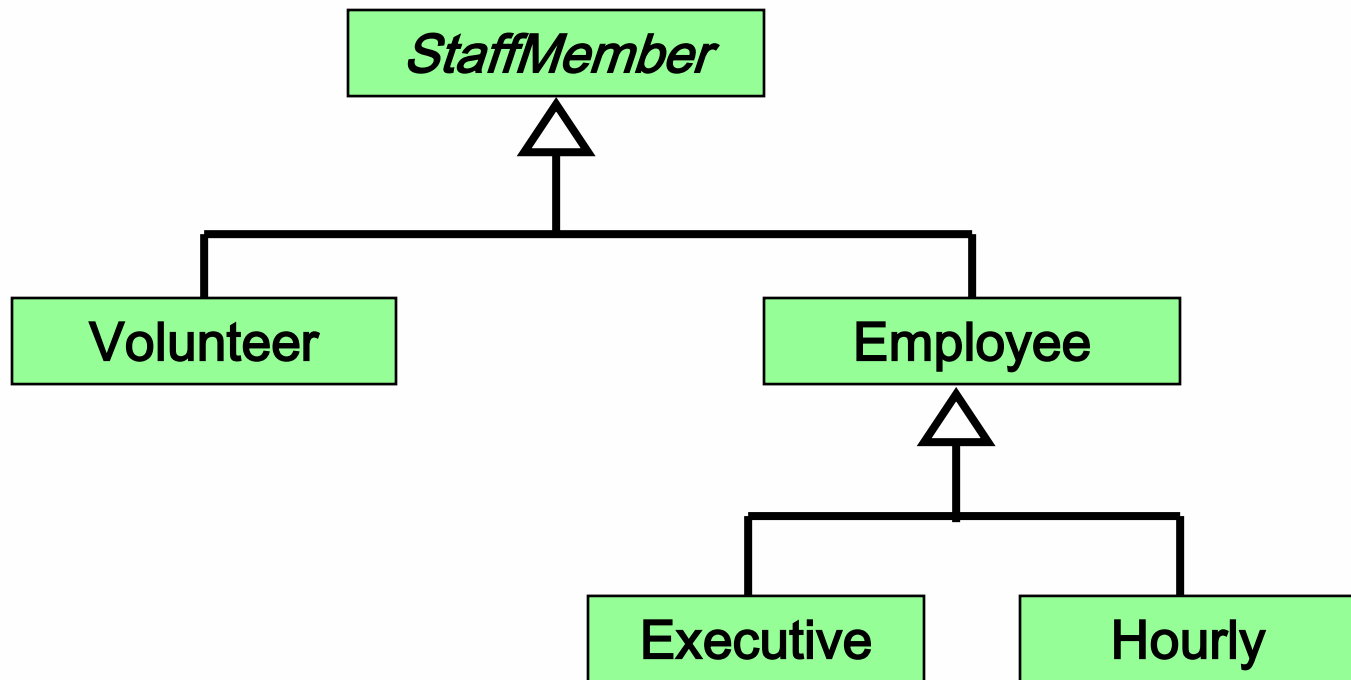
- It is the type of the object being referenced, not the reference type, that determines which method is invoked
- Suppose the `Day` class has a method called `celebrate`, and the `Holiday` class overrides it
- Now consider the following invocation:

```
day.celebrate();
```

- If `day` refers to a `Day` object, it invokes the `Day` version of `celebrate`; if it refers to a `Holiday` object, it invokes the `Holiday` version

Polymorphism via Inheritance

- Consider the following class hierarchy:



Polymorphism via Inheritance

- Now let's look at an example that pays a set of diverse employees using a polymorphic method
- See [Firm.java](#) (page 486)
- See [Staff.java](#) (page 487)
- See [StaffMember.java](#) (page 489)
- See [Volunteer.java](#) (page 491)
- See [Employee.java](#) (page 492)
- See [Executive.java](#) (page 493)
- See [Hourly.java](#) (page 494)

StaffMember.java

```
abstract public class StaffMember
{
```

```
    protected String name;
    protected String address;
    protected String phone;
```

```
    //-----
    //  Constructor: Sets up this staff member using the
    //  specified information.
    //-----
```

```
    public StaffMember (String eName, String eAddress,
                        String ePhone)
    {
        name = eName;
        address = eAddress;
        phone = ePhone;
    }
```

StaffMember.java

```
//-----  
//  Returns a string including the basic employee  
//  information.  
//-----  
public String toString()  
{  
    String result = "Name: " + name + "\n";  
  
    result += "Address: " + address + "\n";  
    result += "Phone: " + phone;  
  
    return result;  
}  
  
//-----  
//  Derived classes must define the pay method for each type  
//  of employee.  
//-----  
public abstract double pay();  
}
```

Volunteer.java

```
public class Volunteer extends StaffMember
{
    //-----
    //  Constructor: Sets up this volunteer using the specified
    //  information.
    //-----
    public Volunteer (String eName, String eAddress,
                     String ePhone)
    {
        super (eName, eAddress, ePhone);
    }

    //-----
    //  Returns a zero pay value for this volunteer.
    //-----
    public double pay()
    {
        return 0.0;
    }
}
```


Employee.java

```
public class Employee extends StaffMember
{
    protected String socialSecurityNumber;
    protected double payRate;

    //-----
    //  Constructor: Sets up this employee with the specified
    //  information.
    //-----
    public Employee (String eName, String eAddress,
                     String ePhone, String socSecNumber,
                     double rate)
    {
        super (eName, eAddress, ePhone);

        socialSecurityNumber = socSecNumber;
        payRate = rate;
    }
}
```


Employee.java

```
//-----  
//  Returns information about an employee as a string.  
//-----  
public String toString()  
{  
    String result = super.toString();  
  
    result += "\nSocial Security Number: " +  
        socialSecurityNumber;  
  
    return result;  
}  
  
//-----  
//  Returns the pay rate for this employee.  
//-----  
public double pay()  
{  
    return payRate;  
}  
}
```

Executive.java

```
public class Executive extends Employee
{
    private double bonus;

    //-----
    //  Constructor: Sets up this executive with the specified
    //  information.
    //-----
    public Executive (String eName, String eAddress,
                     String ePhone, String socSecNumber,
                     double rate)
    {
        super (eName, eAddress, ePhone, socSecNumber, rate);

        bonus = 0;  // bonus has yet to be awarded
    }
}
```

Executive.java

```
//-----  
//  Awards the specified bonus to this executive.  
//-----  
public void awardBonus (double execBonus)  
{  
    bonus = execBonus;  
}  
  
//-----  
//  Computes and returns the pay for an executive, which is  
//  the regular employee payment plus a one-time bonus.  
//-----  
public double pay()  
{  
    double payment = super.pay() + bonus;  
  
    bonus = 0;  
  
    return payment;  
}  
}
```

Hourly.java

```
public class Hourly extends Employee
{
    private int hoursWorked;

    //-----
    //  Constructor: Sets up this hourly employee using the
    //  specified information.
    //-----
    public Hourly (String eName, String eAddress, String ePhone,
                   String socSecNumber, double rate)
    {
        super (eName, eAddress, ePhone, socSecNumber, rate);

        hoursWorked = 0;
    }
}
```

Hourly.java

```
//-----  
//  Adds the specified number of hours to this employee's  
//  accumulated hours.  
//-----  
public void addHours (int moreHours)  
{  
    hoursWorked += moreHours;  
}  
  
//-----  
//  Computes and returns the pay for this hourly employee.  
//-----  
public double pay()  
{  
    double payment = payRate * hoursWorked;  
  
    hoursWorked = 0;  
  
    return payment;  
}
```

Hourly.java

```
//-----  
// Returns information about this hourly employee as a  
// string.  
//-----  
public String toString()  
{  
    String result = super.toString();  
  
    result += "\nCurrent hours: " + hoursWorked;  
  
    return result;  
}  
}
```

Staff.java

```
public class Staff
{
    private StaffMember[] staffList;

    //-----
    //  Constructor: Sets up the list of staff members.
    //-----
    public Staff ()
    {
        staffList = new StaffMember[6];

        staffList[0] = new Executive ("Sam", "123 Main Line",
            "555-0469", "123-45-6789", 2423.07);

        staffList[1] = new Employee ("Carla", "456 Off Line",
            "555-0101", "987-65-4321", 1246.15);
        staffList[2] = new Employee ("Woody", "789 Off Rocker",
            "555-0000", "010-20-3040", 1169.23);

        staffList[3] = new Hourly ("Diane", "678 Fifth Ave.",
            "555-0690", "958-47-3625", 10.55);
```

Staff.java

```
staffList[4] = new Volunteer ("Norm", "987 Suds Blvd.",  
    "555-8374");  
staffList[5] = new Volunteer ("Cliff", "321 Duds Lane",  
    "555-7282");  
  
((Executive)staffList[0]).awardBonus (500.00);  
  
((Hourly)staffList[3]).addHours (40);  
}
```


Staff.java

```
//-----  
// Pays all staff members.  
//-----  
public void payday ()  
{  
    double amount;  
  
    for (int count=0; count < staffList.length; count++)  
    {  
        System.out.println (staffList[count]); // polymorphic  
  
        amount = staffList[count].pay(); // polymorphic  
  
        if (amount == 0.0)  
            System.out.println ("Thanks!");  
        else  
            System.out.println ("Paid: " + amount);  
  
        System.out.println ("-----");  
    }  
}
```

Firm.java

```
//*****  
//  Firm.java          Author: Lewis/Loftus  
//  
//  Demonstrates polymorphism via inheritance.  
//*****  
  
public class Firm  
{  
    //-----  
    //  Creates a staff of employees for a firm and pays them.  
    //-----  
    public static void main (String[] args)  
    {  
        Staff personnel = new Staff();  
  
        personnel.payday();  
    }  
}
```

Output

Name: Sam
Address: 123 Main Line
Phone: 555-0469
Social Security Number: 123-45-6789
Paid: 2923.07

Name: Carla
Address: 456 Off Line
Phone: 555-0101
Social Security Number: 987-65-4321
Paid: 1246.15

Name: Woody
Address: 789 Off Rocker
Phone: 555-0000
Social Security Number: 010-20-3040
Paid: 1169.23

Name: Diane
Address: 678 Fifth Ave.
Phone: 555-0690
Social Security Number: 958-47-3625
Current hours: 40

Paid: 422.0

Name: Norm
Address: 987 Suds Blvd.
Phone: 555-8374
Thanks!

Name: Cliff
Address: 321 Duds Lane
Phone: 555-7282
Thanks!

Outline

Polymorphic References

Polymorphism via Inheritance



Polymorphism via Interfaces

Polymorphism via Interfaces

- An interface name can be used as the type of an object reference variable

```
Speaker current;
```

- The `current` reference can be used to point to any object of any class that implements the `Speaker` interface
- The version of `speak` that the following line invokes depends on the type of object that `current` is referencing

```
current.speak();
```

Polymorphism via Interfaces

- Suppose two classes, `Philosopher` and `Dog`, both implement the `Speaker` interface, providing distinct versions of the `speak` method
- In the following code, the first call to `speak` invokes one version and the second invokes another:

```
Speaker guest = new Philosopher();  
guest.speak();  
guest = new Dog();  
guest.speak();
```